Interpreting Memorial Requirements as Lifecycle Rules: A Design Case of AI-Assisted Construction

Alex Chengyu Li

October 2026

Abstract

Interactive digital memorials require decisions about injury, recovery, absence, and removal. How do these decisions become revisable, inspectable behavior? This retrospective human-computer interaction (HCI) design case follows a technical maker's collaboration with a bereaved pet owner to build a game companion with AI assistance. Dialogue and versioned code reconstruct a revision sequence. A request to avoid death presentation initially became retreat and compulsory rest after any injury. Later feedback narrowed that response to lethal events, requested ordinary healing, and introduced manual release from rest. An administrative request changed removal into quiet disappearance and clearing the saved player–companion recall link. The case distinguishes avoiding death presentation, availability for outdoor companionship, and retention of recovery information. The earlier and corrected injury responses both protected against death but interrupted companionship differently. Record-clearing removal and an archive that retained recovery information both removed the companion, with different later recovery possibilities. A subsequent generic prototype separates lifecycle behavior from a replaceable appearance. Source comparison and recorded scenario checks connect its implementation to the historical rules; resource-reload tests examine appearance replacement. The contribution is an implemented game-case account of consequential interpretation choices, supported by a later generic implementation that can be inspected separately from the personal likeness. Personal discontinuation, managing the shared companion, and disposing of recovery records involve different actions and roles. The case motivates further research on how collaborators choose these consequences and distribute responsibility during AI-assisted making.

1 Introduction

A remembered pet represented in a game can encounter rules that allow injury, retreat, recovery, or removal. These rules determine what encounters remain possible. A brief that asks for protection from death leaves practical choices open: does every injury interrupt play, who can end a recovery pause, and does an administrative removal initiate another return? Answering these questions gives a memorial intention an executable form.

Here, an *interactive digital memorial* is a digital representation intended to sustain a relationship with something remembered through interaction. *Lifecycle rules* determine transitions affecting its continued availability: the triggering event, the action taken, who can initiate or reverse it, and what information persists. The study concerns an extension to the block-based game Minecraft.[1] Its companion was built with AI coding assistance; interaction during play is governed by programmed behavior.

[1] Minecraft: `https://www.minecraft.net/`.

The research question is: *How did memorial requirements become revisable lifecycle rules, and how can their implemented consequences be inspected?* I examine one collaborative construction sequence in which a broad protective proposal was accepted, its treatment of ordinary injury was subsequently corrected, and administrative removal acquired a different outcome. A later generic prototype makes the programmed consequences available for technical inspection independently of the personal likeness. The contribution is a situated example showing how avoiding death presentation, keeping outdoor companionship available, and preserving recovery information can diverge during implementation. Ethical questions concern how collaborators choose among these consequences and how different roles control different resources.

I was the technical maker and researcher. My collaborator owned the deceased pet. I analyze the construction process through my engineering participation. This analysis was undertaken after development. The case is anonymized, with personal circumstances limited to the context needed to understand the memorial requirement.

2 Related Work and the Case's Scope

Ueda and colleagues classify digital memorials and discuss designer, creator, and visitor roles, configuration, volition, and manageability [3]. Their analysis makes control an established concern in memorial design. The present case examines how particular control choices were proposed, revised, and implemented during making.

Afterglow studies AI-mediated pet remembrance through a memorial environment involving generated messages, absence and return, dormancy, deletion, and user-initiated departure [2]. Its lifecycle distinctions are a close predecessor. Some exit mechanisms remain design proposals in that account. In the present case, AI assisted construction, and the relevant game behavior was implemented as deterministic code. The comparison concerns a particular game tension: retaining ordinary injury while protecting against lethal termination and allowing authorized removal. The general importance of voluntary exit is inherited from earlier work.

Liu and colleagues examine how end-user programmers inspect and revise natural-language-to-code interpretations [4]. Their spreadsheet interface turns generated code into stable, editable operation descriptions that users can compare with their requests. This establishes interpretation and correction as existing problems in AI-assisted programming. Our case concerns the consequences of interpreting a loss-linked request as a particular interruption policy, applying established interpretation and correction mechanisms in a memorial setting.

Autobiographical design grounds learning in a maker's own use [1]. Here, technical making and pet bereavement belong to different collaborators. I use a retrospective design-case approach to examine development instructions and implementation choices.

3 Case and Analytic Approach

The artifact is a private Minecraft companion mod, built for game version 1.21.1 with the NeoForge extension framework.[2] Development took place on September 29–30, 2026; the retained final release is 1.2.1. Table 1 summarizes the construction decisions. A technical maker, an owner collaborator, an AI coding assistant, and existing build and test facilities formed the construction arrangement. Minecraft supplied the world, entity system, rendering, sound, and multiplayer infrastructure.

The analysis uses development dialogue, versioned code, and retained technical checks. I

[2]NeoForge: `https://neoforged.net/`.

selected the connected sequence concerning injury, recovery rest, and administrative removal because it explicitly links a memorial constraint to subsequent behavioral revisions. Historical appearance and toy iterations supply project context. Selection is purposeful and retrospective; this sequence provides a bounded account of interpretation during one commission.

I reconstructed each consequential step by locating the instruction, the assistant's proposal or response, and the corresponding behavior in earlier and later code. Table 1 summarizes the sequence. Dialogue establishes what was proposed or directed. Source comparison establishes the implemented transition. Historical tests support only their recorded versions and scenarios. These are distinct kinds of evidence.

The unit of analysis is a development decision and its implemented consequence. The development dialogue records instructions and acceptance messages under one chat account. I use that development account as the attribution unit; individual human authorship is left unspecified. My participation informs the selection and reconstruction. The materials comprise development dialogue and software artifacts; owner experience is a separate topic for use research. Appendix A supplies anonymized paraphrases of the central directions.

In a subsequent engineering phase, I prepared a generic implementation with a newly generated appearance and replaced the private version's identifying names. Section 5 treats this prototype as later technical material. I compare the lifecycle source paths and examine recorded software checks to establish which consequences it retains. The historical sequence establishes how the requirements changed; the generic implementation supports inspection of the resulting rules. It is a later artifact from the same project, with a separate appearance and technical evaluation.

4 Reconstructing the Lifecycle Decisions

4.1 A protective brief became a broad interruption rule

The initial lifecycle request connected the pet's death to a wish to avoid direct death in the game while preserving game balance. Returning home was offered as one possible response, with the assistant invited to develop a suitable arrangement. This set a memorial constraint and a gameplay constraint while leaving the response open.

The assistant proposed retreat before the death sequence, followed by a fixed recovery rest during which outdoor recall would be unavailable. Feeding would not immediately bypass rest. The proposal included environmental dangers and resource restrictions intended to limit gameplay advantages. A subsequent account message accepted proceeding with this plan.

The retained earlier implementation shows the scope that this interpretation acquired. When the game software requested an amount of damage greater than zero, the code initiated return and recovery rest without lowering health. Software requests to invoke death or removal entered the same recovery route. This made even an ordinary injury event an interruption.

4.2 Later feedback separated ordinary injury from compulsory rest

Later feedback explicitly described rest after every injury as too frequent. It requested automatic healing after ordinary injury, compulsory rest only on the death pathway, and a manual release control. The assistant acknowledged the change and subsequently identified the existing every-injury rest behavior. The dialogue connects a reported practical problem to a revised instruction and a matching technical diagnosis.

The code revision allowed ordinary health reduction followed by delayed healing. Lethal damage and below-world failure continued to initiate return and recovery. The game records a player

as the companion's owner. That player gained an action to end rest after the companion had arrived, with unresolved return still awaiting a safe destination. This game-level role is distinct from being the deceased pet's real-world owner. The correction changed both the extent of interruption and control over its duration.

This is the key learning episode. The accepted protective proposal was subsequently reopened to distinguish which nonterminal events should interrupt play. The revision preserved an injury response while restricting compulsory absence. The account's correction supplies a reason for that narrowing: the earlier response interrupted too frequently.

Table 1: Reconstructed decisions and their evidential scope. Directions are attributed to the development account; code establishes implementation.

Step	Direction and disposition	Evidence-supported consequence
P1	Avoid direct death while preserving balance; home return is a possible response.	A loss-linked constraint is explicit; the complete event policy is still open.
P2	Assistant proposes retreat and fixed rest; account accepts proceeding.	Earlier code sends software requests to apply damage or remove the companion into recovery.
P3	Account reports excessive rest and requests ordinary healing plus manual release.	Later code separates health loss from lethal recovery and adds release after arrival.
P4	Account requests gentle administrative cleanup, then standard removal behavior with existing controls retained.	Removal clears the in-world companion and saved player–companion recall link, independently of lethal recovery.

4.3 Administrative removal acquired a different consequence

A subsequent instruction requested quiet cleanup of an unowned companion by a server operator. The assistant proposed an operator-only interface. The account then requested that the game's standard removal operation use the same gentle handling, briefly favoring fewer dedicated controls, and finally directed that the existing controls be retained. This sequence revised the administrative interface and its behavior.

Recall uses a stored link from a player's record to a particular game companion, together with saved information for finding or restoring that companion. Earlier code treated the removal method as another recovery trigger. Revised code quietly removed the current companion from the world and cleared the link and its recovery information. Subsequent recall through that link was unavailable. Lethal injury continued to initiate recovery. Avoiding a death presentation consequently coexisted with two different outcomes for what remained stored. Calls to the removal method can originate in administration or software. Personal intention requires separate attribution to an identified request and its authorization.

For a companion linked to a player's record, an existing operator-authorized archive route quietly removed it from the world while retaining its link and restoration information. These stored records allowed a separate restoration request. Restoration first requires confirmation, then searches for the existing companion; recreation after unsuccessful search depends on retained information and loading conditions. Removal and archive therefore differ in what can subsequently be recovered. The distinction concerns the in-world companion and its associated saved records; independent server backups retain their own histories.

4.4 Availability also depended on technical execution

The account reported that recall sometimes announced arrival while a player could hear the companion but could not see it; another player could see it. The requested correction was shared visibility. This distinguishes a rule about returning from successful access to the returned representation. Such an implementation failure could interrupt an encounter even when the intended lifecycle policy preserved it.

Dedicated-server/two-client checks in version 1.2.0 examine whether the companion's state is shared consistently. They supply version- and scenario-specific observations of return behavior.

5 A Generic Implementation for Inspecting the Rules

The later prototype couples the retained companion behavior to a resource-defined appearance. Its default geometry and textures depict a generic grey-and-white dog, shown in Figure 1. A local generation program specifies the cubes, joint positions, texture coordinates, and pixels. A client resource pack can replace the appearance while the server retains the companion's injury, recovery, ownership, and removal logic. This separation permits technical work with a substitute representation while personal images and the original likeness remain private.

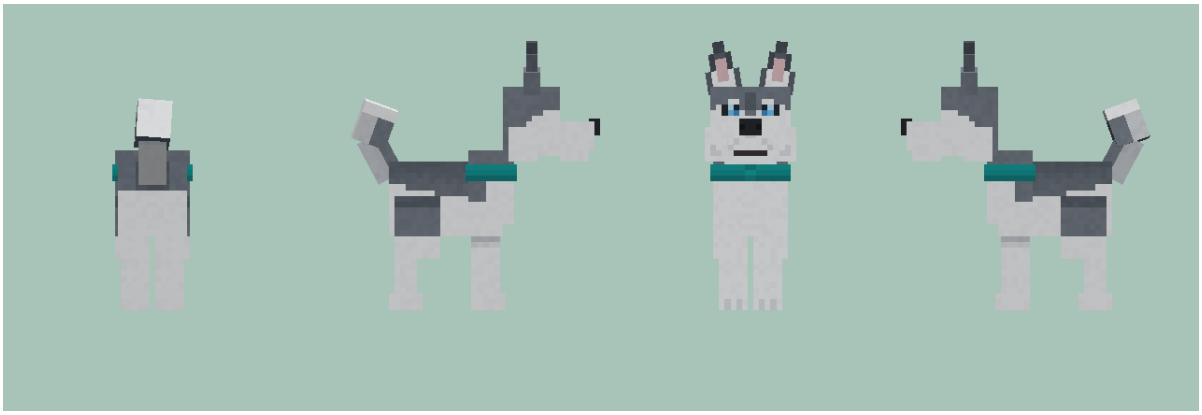


Figure 1: The later generic prototype, rendered from four directions by the game client. This newly generated appearance is used to inspect the implementation separately from the private case's personal likeness. The image documents the standing rendering; animation compatibility is evaluated separately.

Table 2 summarizes consequences that can be followed through the implementation. Ordinary injury can reduce health and schedule healing without compulsory return. Lethal damage and below-world failure initiate protected return and recovery. The recorded game owner can end rest after arrival. Standard removal clears the recall link and recovery information; the operator archive preserves records for a separate, conditional restoration request. These are specific implemented rules with distinct triggers, controls, and retained resources.

The technical comparison has two parts. After replacing identifying names, the lifecycle entity, saved-association, safety, and administrative source files match the corrected historical implementation. The generic prototype uses its own player associations and saved data. Recorded scenarios in the generic implementation exercise ordinary and lethal injury, owner rest release after arrival, rejection of another player's request, and removal with recall-link clearing. The archive and conditional-restoration distinction is supported by the inspected source paths. This gives the comparison a code-level basis and tests specific consequences in the generic artifact. A matched execution of both versions under an identical protocol would supply a further test

Table 2: Lifecycle consequences in the corrected implementation. The table separates events, immediate responses, authorized controls, and retained information.

Event	Immediate response	Control	Retained information
Ordinary injury	Health loss and delayed healing	Programmed response	Recall link and recovery records remain.
Lethal danger	Protected return and recovery rest	Programmed response; safe arrival required	Companion remains associated with its recorded owner.
End rest	Resume availability after arrival	Recorded game owner	Association and recovery information remain.
Standard removal	Quiet world removal	Authorized administration or software call	Recall link and associated recovery records are cleared.
Operator archive	Quiet world removal with checkpoint	Confirmed operator action on a linked companion	Records support a separate, conditional restoration request.

of behavioral preservation.

Resource-reload checks separately examine replacement through a directory pack and a ZIP pack, pack priority, an empty rig, invalid definitions or textures, and restoration of the default appearance. They confirm that appearance replacement reaches a rebuilt renderer and that invalid input yields empty rendering. Geometry inspection also identified ear and tail ground penetration during some movements. Appearance substitution therefore retains an asset-specific calibration task. The companion repository supplies the generic source, appearance generator, and scenario-test source.[3] The canonical build requires separately installed testing infrastructure that is not distributed with this repository; the technical observations retain their recorded local setup.

6 Analysis: Presentation, Interruption, and Recovery

Avoiding death presentation and keeping outdoor companionship available had different consequences in the inspected code. The earlier implementation protected the companion from death while making ordinary injury initiate return and compulsory rest. The correction retained protection from lethal termination while allowing ordinary health loss without that forced return. The game's recorded owner also gained an action to end rest after arrival. Thus two responses compatible with the no-direct-death brief interrupted companionship differently.

Quiet disappearance and recoverability likewise differed. Revised removal and the existing operator archive both removed the current companion from the world. Removal cleared the saved recall link and recovery information; archive retained them for a separate, conditional restoration request. Their shared quiet presentation therefore accommodated different later access to the same companion. These contrasts distinguish the presentation of an event, the activity it interrupts, and the information that survives it.

The revisions involved refining requirements, separating event handlers, and implementing authorization. Feedback can clarify an earlier preference, change a preference, or respond to a new consequence; these possibilities remain open in the reconstruction. The analytical result concerns the divergence documented within this commission. The same presentation constraint accommodated different interruption rules, and similarly quiet removal accommodated different

[3]Companion Dog Framework, implementation snapshot: `https://github.com/crabsatellite/CompanionDogFramework/tree/632dc7083b153fed6e4a975fdecc8ea57bd19a26`.

recoverability. This example is useful when death avoidance or gentle removal is treated as a complete description of a memorial's availability. Assessing the intended encounter also requires attending to the implemented interruption and stored recovery consequences.

The related work addresses return, departure, retained information, role allocation, and request-to-code correction. This case adds an implemented, revisable game-instance example in which a death-avoiding brief acquired different injury and removal policies. Its contribution lies in the concrete divergence of presentation, interruption, and recoverability exposed by those revisions.

AI participated in proposing and implementing the first response and in making the correction. Proposed behavior, executable changes, and later revisions occurred within the same conversational construction process. The technical collaborator could direct that process while inspecting and testing the resulting game object. This arrangement connects personal memorial requests to a modifiable artifact through human judgment, AI assistance, and existing game infrastructure. Comparing those arrangements can examine how responsibility and interpretation are distributed during making.

The generic prototype adds a way to examine the programmed consequences apart from the personal representation. The same lifecycle questions can be investigated with a substitute appearance, while likeness and personal meaning remain matters for the original collaboration. This separation supports inspection of what the program does and preserves the distinction between an engineering demonstration and the remembered companion's significance to its owner.

7 Design Implications and Limits

The reported interruption problem supplies a practical reason to surface relevant consequences when collaborators choose or revise a lifecycle arrangement. For this brief, examples include ordinary damage causing forced return, the available rest-release action, and whether the selected route retains or clears recovery information. This consideration depends on the collaborators' purposes and the situation. Initial expectations, how much detail the collaborators wanted, and whether advance explanation would have prevented the revision remain questions for inquiry. Whether explanation is an ethical duty requires a separate argument about legitimate choices, foreseeable consequences, delegation, competing interests, and proportionate burdens.

A person may deliberately choose continuous presence and automatic return, regard closing the game or uninstalling as sufficient discontinuation, or knowingly delegate routine details and request changes later. Those possibilities limit both a compulsory ending interface and a uniform requirement for detailed advance explanation. The practical proposal concerns relevant choices during an actual commission, with detail and revision arrangements responsive to the collaborators' purposes. The case records a later request for correction; the person's initial understanding and preferred delegation arrangement remain unresolved.

Shared environments involve distinct endpoints: ending personal interaction, managing a shared game instance, and retaining or clearing recovery information. Closing the game may end a person's interaction while leaving the server companion and its saved records intact; this may be sufficient for that person's purpose. An operator may manage the shared instance for other players or world maintenance. Clearing recovery records changes a further resource. The observed operator request leaves the appropriate allocation among these roles open. Calls to a removal method establish software consequences; a personal decision to end companionship requires separate attribution. These distinctions limit the proposal to considering particular roles, actions, and resources, with the exit mechanism and competing priorities open to selection.

Technical execution and lived experience pose different evaluation questions. Shared visibility makes return accessible; use research can investigate whether that return is welcome. A consented, rule-specific study could follow the owner's understanding of transitions, changing preferences, and decisions to continue or discontinue interaction.

The larger research agenda concerns who can commission and revise a personally meaningful interactive memorial, and what support makes that process feasible. This case realizes one arrangement combining a technical collaborator, conversational AI assistance, and game infrastructure. Comparative work could hold requirements and acceptance conditions explicit across AI-assisted making, configurable templates, and conventional development, recording skill, human work, rework, verification, and maintenance. Those observations would assess how AI changes the threshold for personal memorial construction.

This purposefully selected sequence offers construction knowledge from one collaboration. Researcher participation and access to private materials shape the analysis, while historical technical observations retain their version and scenario scope. The later prototype supplies technical material from the same project. Its source comparison and scenario checks concern programmed consequences; the meaning of the original likeness, sustained use, and comparative creation effort require further inquiry. Further cases can examine how the identified lifecycle choices vary across creators and construction arrangements.

8 Conclusion

This construction sequence documents a divergence between avoiding death presentation, keeping outdoor companionship available, and retaining information for later recovery. The divergence became inspectable through an accepted proposal, a correction of excessive rest, and a separate removal revision. A later generic implementation connects these rules to source paths and scenario checks while separating them from the private personal likeness. The case supplies a concrete example for evaluating lifecycle consequences and a practical reason to discuss them when choosing or revising an arrangement. Further research can examine proportionate explanation, delegation, and authority over the different endpoints.

Data and AI Assistance

The technical maker and pet owner collaborated on the original artifact. Raw dialogue, personal images, and identifying material remain private. The collaborator permitted the anonymized case description and paraphrases reproduced here. The later generic appearance is separate technical material. Generic source and the asset-generation program accompany the paper in the repository identified in Section 5; identifying technical records remain private.

OpenAI Codex was used as the AI-agent interface in construction and manuscript preparation, including implementation, asset generation, drafting, translation, and critical reading. I reviewed the resulting material and take responsibility for the analysis, attribution, and manuscript.

A Selected Anonymized Directions

The following concise paraphrases of development-account directions preserve the content used to reconstruct the decisions.

P1. Avoid the companion's direct death while retaining game balance; automatic home return is one possible response, open to further design.

P2. Proceed with the proposed arrangement and report after the system is ready for player testing.

P3. Rest should be manually releasable and reserved for the death pathway. Ordinary injury should heal automatically; resting after every injury happens too frequently.

P4. Permit operators to quietly remove an unowned companion. Apply gentle handling to the standard removal operation; retain the previous controls.

References

[1] C. Neustaedter and P. Sengers. 2012. Autobiographical design in HCI research: designing and learning through use-it-yourself. *DIS 2012*, 514–523. `https://clab.iat.sfu.ca/pubs/AutobiographicalDesign.pdf`.

[2] H. Shi and D. DiFranzo. 2026. *Afterglow: A Place-Based Memorial Ecology for AI-Mediated Pet Bereavement*. Author preprint, arXiv:2609.38729v1. `https://arxiv.org/abs/2609.38729v1`.

[3] G. Ueda, L. F. F. Monteiro, C. Maciel, and V. C. Pereira. 2022. Digital memorials: classifications and design recommendations. *Journal on Interactive Systems* 13(1), 335–349. `https://doi.org/10.5753/jis.2022.2567`.

[4] M. X. Liu, A. Sarkar, C. Negreanu, B. Zorn, J. Williams, N. Toronto, and A. D. Gordon. 2023. What It Wants Me To Say: Bridging the Abstraction Gap Between End-User Programmers and Code-Generating Large Language Models. *CHI 2023*. `https://doi.org/10.1145/3544548.3580817`.

First version publicly available: 12 October 2026. Paper archive: `https://doi.org/10.5281/zenodo.23320935`.